Transaktionen

10 Vertiefungspunkte in der Reihenfolge der Originalfragen

01 **Fehlerfälle.** Programmfehler, System- oder Hardwareausfälle, Netzwerkabbrüche und verletzte Regeln wie doppelte Schlüssel oder ungültige Fremdschlüssel können einen mehrstufigen Schreibvorgang unterbrechen. Bei einem SQL-Fehler muss die Anwendung bei Bedarf ausdrücklich zurückrollen. Ein einzelner Fehler setzt nicht zwingend die gesamte Transaktion zurück.

02 **Gleichzeitige Zugriffe.** Isolation und Sperren verhindern je nach Stufe, dass sich parallele Transaktionen gegenseitig stören. Ein *Dirty Read* liest unbestätigte Daten. Beim *nicht wiederholbaren Lesen* hat dieselbe Zeile beim zweiten Lesen einen anderen Wert.

03 **Transaktion.** Eine Gruppe zusammengehöriger SQL-Operationen bildet eine Einheit. Mit `START TRANSACTION` beginnen, bei Erfolg mit `COMMIT` bestätigen und bei Fehler mit `ROLLBACK` zurücknehmen.

04 **ACID. A**tomarität bedeutet alles oder nichts. **C**onsistency bedeutet, dass die Integritätsregeln nach erfolgreichem Abschluss erfüllt sind. **I**solation regelt, was parallele Transaktionen voneinander sehen. **D**urability bedeutet, dass bestätigte Änderungen einen Absturz überstehen.

05 **Isolationsebenen.** Nach SQL-Standard gelten folgende Mindestgarantien. Höhere Isolation kann Parallelität kosten.

Stufe	Verhindert mindestens
0 · `READ UNCOMMITTED`	Keines der drei genannten Leseprobleme
1 · `READ COMMITTED`	Dirty Reads
2 · `REPEATABLE READ`	Zusätzlich nicht wiederholbares Lesen. Standard in MariaDB
3 · `SERIALIZABLE`	Zusätzlich Phantome

06 **Phantome.** Dieselbe Abfrage liefert beim zweiten Mal eine andere Zeilenmenge. A zählt Filme eines Genres. B fügt einen passenden Film ein und bestätigt. Auf einer Stufe, die Phantome zulässt, kann A nun einen Film mehr zählen. Auch Löschungen können die Treffermenge ändern.

07 **Portalbeispiel.** Film einfügen, seine ID mit `LAST_INSERT_ID()` holen und alle Genrezuordnungen schreiben. Scheitert die zweite Zuordnung, wird auch der Film zurückgerollt. Weitere Fälle sind das Ersetzen von Zuordnungen oder das gemeinsame Speichern einer Änderung und ihres Logeintrags.

08 `COMMIT`**.** Beendet die Transaktion und bestätigt ihre Änderungen dauerhaft. Andere Transaktionen sehen sie gemäß ihrer Isolation. Auch reine Lesetransaktionen abschließen, damit etwaige Sperren freigegeben werden.

09 `ROLLBACK`**.** Beendet die Transaktion und nimmt ihre unbestätigten Änderungen zurück. Für gezieltes Zurückrollen bis zu einem Zwischenstand braucht man einen `SAVEPOINT`. Nach einem Absturz setzt die Datenbank offene InnoDB-Transaktionen zurück.

10 **Speicher-Engine.** In den Beispielen braucht man InnoDB. Es unterstützt Transaktionen, Zeilensperren und Fremdschlüssel. MyISAM-Änderungen lassen sich nicht zurückrollen. Prüfen mit `SHOW ENGINES;` und `SHOW TABLE STATUS FROM wer_streamt_was;`. Eigene Tabellen lassen sich mit `ALTER TABLE film ENGINE = InnoDB;` umstellen. MariaDB-Systemtabellen nicht umwandeln.

Stored Procedures

10 Vertiefungspunkte in der Reihenfolge der Originalfragen

01 **Definition.** Benanntes Unterprogramm mit SQL und Ablaufsteuerung, gespeichert und ausgeführt auf dem Datenbankserver. Es kann Parameter annehmen und Ergebnisse zurückgeben.

02 **Einsatz.** Wiederkehrende Abläufe mit verschiedenen Eingaben, mehreren SQL-Anweisungen oder großen Datenmengen. Beispiele sind eine Überweisung oder das Anlegen eines Films mit Genre über einen einzigen Aufruf aus PHP.

03 **Vorteile.** Weniger Netzwerk-Rundreisen und Datentransfer. Geschäftslogik wird zentral gewartet und kann bei unverändertem Aufruf optimiert werden. Mit passenden Rechten kann ein Konto die Prozedur ausführen, ohne direkten Zugriff auf die Tabellen zu erhalten.

04 `CALL` **und** `SELECT`**.** MariaDB-Prozeduren werden mit `CALL` ausgeführt und können Daten ändern, Ausgabeparameter setzen oder Ergebnismengen liefern. Selektierbare Prozeduren, die wie eine Tabelle in `SELECT` und `JOIN` verwendet werden, unterstützt MariaDB in dieser Form nicht.

05 **Deklarativ und prozedural.** SQL beschreibt das gewünschte Ergebnis. Prozedurales SQL beschreibt den Ablauf mit Variablen, `IF`, Schleifen wie `WHILE` und Fehlerbehandlung per `DECLARE … HANDLER`. Oracle nennt seine Erweiterung PL/SQL, PostgreSQL verwendet PL/pgSQL und MariaDB/MySQL haben einen eigenen Dialekt.

06 `DELIMITER`**.** Im SQL-Client vorübergehend etwa `DELIMITER //` setzen. Sonst schickt der Client eine mehrteilige Definition schon beim ersten inneren Semikolon ab. Die Definition endet mit `END //`. Danach mit `DELIMITER ;` zurückstellen.

07 **Parameter.** `IN` übergibt eine Eingabe, z. B. eine Film-ID, und ist der Standard. `OUT` liefert eine Ausgabe, z. B. eine Anzahl oder neue ID. `INOUT` übergibt einen Wert und gibt ihn verändert zurück. Beispiel für `OUT`: `CALL sp_count_movies(2010, @anzahl);` und danach `SELECT @anzahl;`.

08 **Lokale Variablen.** Am Blockanfang nach `BEGIN` mit `DECLARE v_name DATENTYP [DEFAULT wert];` anlegen, mit `SET` oder `SELECT … INTO` befüllen. Gültig im eigenen und in verschachtelten Blöcken. Präfixe wie `v_` und `p_` sowie qualifizierte Spaltennamen vermeiden Mehrdeutigkeiten wie `WHERE title = title`.

09 **Film mit Genre.** Titel, Jahr und Genre übergeben. Film einfügen, neue ID merken, Genre per `SELECT … INTO` suchen, falls nötig anlegen und Film und Genre in der Zwischentabelle verknüpfen. `AUTO_INCREMENT` vergibt die ID, `LAST_INSERT_ID()` liefert die eigene zuletzt erzeugte ID. `MAX(id) + 1` kann bei parallelen Inserts dieselbe ID liefern.

10 **Verwalten.** `SHOW PROCEDURE STATUS WHERE Db = 'wer_streamt_was';` listet Prozeduren, `SHOW CREATE PROCEDURE name;` zeigt den Code. Code und Parameter mit `CREATE OR REPLACE PROCEDURE` oder Löschen und Neuanlegen ändern. `ALTER PROCEDURE` ändert Eigenschaften wie den Kommentar. Löschen mit `DROP PROCEDURE IF EXISTS name;`.

Trigger

10 Vertiefungspunkte in der Reihenfolge der Originalfragen

01 **Definition.** Ein Trigger läuft automatisch bei `INSERT`, `UPDATE` oder `DELETE` auf seiner Tabelle. Eine Stored Procedure wird mit `CALL` gestartet. Trigger haben keine Aufrufparameter und greifen über `NEW` und `OLD` auf Zeilenwerte zu.

02 **Aufbau.** Name, Auslösebedingung (Zeitpunkt, Ereignis, Tabelle) und Ausführungsteil (SQL, Variablen und Kontrollstrukturen). Grundform: `CREATE TRIGGER name BEFORE INSERT ON tabelle FOR EACH ROW ….`

03 **Einsatz.** Regeln sichern (Bewertung 1 bis 5), abgeleitete Werte vorberechnen (Durchschnitt), fehlende Werte ergänzen und Änderungen in einer Log-Tabelle protokollieren.

04 `BEFORE` **und** `AFTER`**.** `BEFORE INSERT` kann ungültige Bewertungen vor dem Speichern ablehnen oder neue Werte anpassen. `AFTER INSERT` kann den Filmdurchschnitt aktualisieren, weil die neue Bewertung bereits in der Tabelle steht.

05 `FOR EACH ROW`**.** In MariaDB Pflicht. Ein `UPDATE` über fünf betroffene Zeilen startet den Trigger fünfmal, jeweils mit den Werten der betreffenden Zeile.

06 `NEW` **und** `OLD`**.** `INSERT` hat nur `NEW`, `UPDATE` hat beide, `DELETE` nur `OLD`. `NEW.spalte` liest den neuen Wert, `OLD.spalte` den bisherigen. In einem passenden BEFORE-Trigger kann man `SET NEW.spalte = …` verwenden.

07 **Tabellenbezug.** Ein MariaDB-Trigger gehört zu genau einer Tabelle. Zwei Tabellen brauchen je einen Trigger. Beide können gemeinsame Logik per `CALL` nutzen. Mehrere Trigger für dieselbe Tabelle und dasselbe Ereignis sind seit MariaDB 10.2 möglich.

08 **Lagersummen.** `trig_summe_artikel` speichert die Summe aus `t_lager` in `t_summe_lager`. Lesen braucht dann nur eine Zeile. Reagiert der Trigger nur auf `INSERT`, fehlen Änderungen durch `UPDATE` und `DELETE`. Effizienter als jedes Mal neu zu summieren ist eine Anpassung um die Differenz, beim Insert z. B. `summe_stueck + NEW.stueck`.

09 **Eigener Fehler.** Bei `NEW.rating < 1 OR NEW.rating > 5` bricht `SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Bewertung muss zwischen 1 und 5 liegen';` das Insert ab. PDO im Exception-Modus wirft eine `PDOException`. PHP fängt sie mit `try/catch` ab und gibt eine verständliche, für HTML mit `htmlspecialchars()` maskierte Meldung aus.

10 **Trigger finden und ändern.** Ein unerwartetes Update von `average_rating` kann aus einem Trigger auf der Bewertungstabelle kommen. `SHOW TRIGGERS;` listet Trigger, `SHOW CREATE TRIGGER name;` zeigt den Code. Änderungen per `CREATE OR REPLACE TRIGGER` oder `DROP TRIGGER IF EXISTS name;` und Neuanlegen. Die Logik dokumentieren, damit sie bei der Fehlersuche auffindbar ist.

SQL-Benutzerverwaltung

10 Vertiefungspunkte in der Reihenfolge der Originalfragen

01 **Host beschränken.** Ein Konto wie `'portal_user'@'localhost'` ist für lokale Verbindungen gedacht. `'%'` kann Verbindungen von beliebigen Hosts zulassen, die den DB-Server erreichen. Lokal begrenzen verkleinert die Angriffsfläche.

02 **SQL-Injection und Rechte.** Ein eingeschränktes `SELECT`-Konto kann keine Tabellen löschen oder Daten ändern. Angreifer können aber im Rahmen seiner Leserechte sensible Daten abfragen. Ein weitreichendes Konto kann je nach Rechten und Angriff auch `DROP` oder `ALTER` ermöglichen. Prepared Statements verhindern, dass gebundene Eingaben zu SQL-Code werden.

03 **DB-Konten und Portalnutzer.** Wenige DB-Konten passend zu den App-Rollen genügen. Die Anwendung ordnet Abfragen der Person aus der Session zu, z. B. `SELECT … FROM watchlist WHERE user_id = ?`.

04 **Minimale Rechte.** App-Konten erhalten nur die benötigten Rechte, etwa `SELECT`, `INSERT`, `UPDATE` oder `DELETE` auf bestimmte Objekte. `ALL PRIVILEGES` gilt für den angegebenen Bereich, etwa `*.*` für die Administration oder `wer_streamt_was.*` für ein separates Entwicklungskonto.

05 `USAGE`**.** In `SHOW GRANTS` bedeutet `GRANT USAGE ON *.*`, dass das Konto durch diesen Eintrag keine Objektrechte erhält. Die Anmeldung ist mit gültigen Zugangsdaten möglich.

06 **View für Gäste.** Eine gespeicherte Abfrage wie `CREATE VIEW v_public_filme AS SELECT id, titel, beschreibung, genre_id FROM filme WHERE veroeffentlicht = 1;` gibt nur die vorgesehenen Zeilen und Spalten frei. Das Gastkonto erhält `SELECT` ausschließlich auf diese Sicht. Das vereinfacht Abfragen und schützt interne Felder.

07 **Rechte entziehen.** `REVOKE INSERT ON wer_streamt_was.filme FROM 'redakteur'@'localhost';` entfernt ein früher vergebenes Recht. Mit `SHOW GRANTS FOR 'redakteur'@'localhost';` kontrollieren. Überflüssige Konten mit `DROP USER` entfernen, damit sich keine unnötigen Rechte ansammeln (*Privilege Creep*).

08 **Anlegen erlauben.** `GRANT SELECT, INSERT ON wer_streamt_was.filme TO 'redakteur'@'localhost';` erlaubt Lesen und Anlegen. `UPDATE` nur bei Bedarf ergänzen. Für ein Löschverbot darf auch über andere Grants oder Rollen kein `DELETE` vorliegen. Gemeinsame Rechte mehrerer Redakteure lassen sich in einer Rolle bündeln.

09 **Zugangsdaten.** Konfiguration außerhalb von `htdocs` oder `/var/www/html` speichern, aus PHP mit `require` laden und per `.gitignore` vom Repository fernhalten. Bei Fehlkonfiguration oder einer öffentlich erreichbaren `.bak`-Datei können Zugangsdaten im Klartext sichtbar werden.

10 **Eigene E-Mail ändern.** Die DB beschränkt das Spaltenrecht mit `GRANT UPDATE (email) ON wer_streamt_was.users TO 'portal_user'@'localhost';`. Die App beschränkt die Zeile per Prepared Statement `UPDATE users SET email = ? WHERE user_id = ?`. Die ID stammt aus der Session. Eine Stored Procedure kann die Update-Logik kapseln.

Backup und Restore

10 Vertiefungspunkte in der Reihenfolge der Originalfragen

01 **Zweck und 3-2-1-Regel.** Regelmäßige Backups sichern Daten gegen Defekte, Fehlbedienung und Angriffe. **3** Kopien einschließlich Original, auf **2** verschiedenen Speichermedien, davon **1** außer Haus. Beispiel für das Portal: Produktivdaten, lokaler Dump und eine Kopie auf einem externen Speichermedium an einem anderen Standort.

02 **Offline und online.** Offline ist der DB-Server gestoppt. Die Dateien sind konsistent, das Portal ist jedoch nicht erreichbar. Online läuft der Server weiter. Damit kein Mischzustand entsteht, Schreibvorgänge per Lesesperre blockieren oder für InnoDB mit `mysqldump --single-transaction` einen konsistenten Snapshot verwenden. Bei einem Snapshot können normale Schreibvorgänge weiterlaufen.

03 **Sperren und Flush.** `READ` erlaubt Lesen und sperrt Schreiben. `READ LOCAL` erlaubt zusätzlich konfliktfreie Inserts bei MyISAM. `WRITE` erlaubt nur der sperrenden Sitzung zu lesen und zu schreiben. Für eine konsistente Dateikopie offene Tabellen und gepufferte Änderungen berücksichtigen. Im Beispiel kombiniert `FLUSH TABLES movie, genre WITH READ LOCK;` Flush und Lesesperre. Danach mit `UNLOCK TABLES;` freigeben.

04 **CSV mit `SELECT … INTO OUTFILE`.** Exportiert Daten ohne Tabellenstruktur. Die Datei entsteht am DB-Server und darf noch nicht existieren. Das DB-Konto braucht `FILE`, der Serverprozess Schreibrechte und der Pfad muss gemäß `secure_file_priv` erlaubt sein. Standard sind Tabulator, keine Textbegrenzung, Backslash zum Maskieren und Zeilenumbruch. Import mit `LOAD DATA INFILE` und denselben Optionen.

```
SELECT * FROM movie
INTO OUTFILE '/var/lib/mysql-files/movie.csv'
FIELDS TERMINATED BY ';' ENCLOSED BY '"'
LINES TERMINATED BY '\n';
```

05 **SQL-Dump.** Textdatei mit SQL, typischerweise `DROP TABLE IF EXISTS`, `CREATE TABLE` und `INSERT INTO … VALUES`. Beim Import werden Struktur und Inhalt neu aufgebaut. Ein Dump ist komprimierbar und meist leichter zwischen kompatiblen Servern zu übertragen als ein Datenverzeichnis, das an Version, Engine und Umgebung gebunden ist.

06 **Verfahren wählen.** Dateikopie auf Betriebssystemebene für das gesamte Datenverzeichnis bei gestopptem Server. `OUTFILE` für Datenaustausch als CSV. `mysqldump` bzw. `mariadb-dump` für regelmäßige logische Sicherungen oder Umzüge. phpMyAdmin für einen gelegentlichen Export per Oberfläche, meist als SQL mit Struktur und Daten.

07 **Sicherheit.** Keine Passwörter im Befehl oder Skript. Eine geschützte Optionsdatei wie `~/.my.cnf` mit `chmod 600` verwenden. Eigenes Konto mit den minimalen Rechten für das gewählte Verfahren. Dumps außerhalb des Webroots lagern und eine Kopie außer Haus behalten.

08 **Automatisierung.** Das Skript erstellt und komprimiert den Dump, vergibt einen eindeutigen Namen mit Datum, erkennt und protokolliert Fehler und entfernt alte Sicherungen nach einer Aufbewahrungsfrist. So laufen Sicherungen regelmäßig und einheitlich ab.

09 **`cron`.** Zeitplan-Dienst unter Linux. Mit `crontab -e` bearbeiten. Die Felder sind Minute, Stunde, Tag des Monats, Monat, Wochentag und Befehl. `*` steht für jeden Wert. `30 2 * * * /usr/local/bin/backup.sh` startet die Sicherung täglich um 02:30 Uhr, wenn das Portal typischerweise wenig genutzt wird.

10 **Restore testen.** Ein Dump kann leer, beschädigt oder unvollständig sein. Im Ernstfall zuerst die Ursache stoppen, dann die Sicherung in eine Testdatenbank einspielen, die Daten prüfen und gezielt die verlorenen Datensätze übernehmen. Ein Restore allein aus dem Dump stellt den Stand zum Sicherungszeitpunkt wieder her. Das Sicherungsintervall bestimmt, wie viele neuere Änderungen fehlen können.

Caching

10 Vertiefungspunkte in der Reihenfolge der Originalfragen

01 **Hit und Miss.** Beim Hit liegen gültige Daten im Cache. Beim Miss fehlt der Eintrag oder ist abgelaufen. Die Anwendung lädt z. B. die Trending-Liste aus der DB, speichert sie und liefert sie aus. Solange der Eintrag gültig ist, trifft die nächste gleiche Anfrage den Cache.

02 **Nutzen und Preis.** Caching verkürzt Ladezeiten, spart Server- und DB-Arbeit und kann bei einem kurzen Ausfall der Quelle weiterhin Daten liefern. Die Kopie kann bis zur Aktualisierung veraltet sein.

03 **Vier Ebenen.** Browser → CDN → Anwendung → Datenbank. Ein früher Treffer spart alle dahinterliegenden Zugriffe. Ein frischer Browser-Treffer kann sogar eine Netzwerkanfrage sparen.

04 **HTTP-Header.** `Cache-Control: public, max-age=604800` erlaubt das Zwischenspeichern in gemeinsam genutzten Caches und setzt die Frische auf 7 Tage. Der `ETag: "poster-1042-v3"` kennzeichnet die Version. Danach fragt der Browser mit `If-None-Match` nach. Unverändert → `304 Not Modified` und alte Kopie weiterverwenden. Geändert → neue Datei mit neuem ETag.

05 **CDN.** Ein *Content Delivery Network* verteilt Kopien auf geografisch verteilte Server. Ein naher Standort senkt die Latenz und entlastet den Ursprungsserver. Geeignet für Trailer, Poster, Thumbnails, Logos, CSS und JavaScript.

06 **Serverseitige Caches.** Ein In-Memory-Cache speichert im RAM, z. B. Trending- oder Genrelisten mit Redis, Memcached, APCu oder `IMemoryCache`. Ein DB-Abfragecache kann bei entsprechender Unterstützung Ergebnisse wiederholter Abfragen halten. Ein Objekt-Cache speichert bereits zusammengesetzte Daten, z. B. ein Filmdetail-Objekt aus mehreren JOINs.

07 **Cache-Aside.** `getTrendingMovies()` prüft, ob `trending.json` existiert und jünger als 600 Sekunden ist. Hit → mit `json_decode()` lesen und zurückgeben. Miss → die zehn meistgesehenen Filme per SQL holen. Ergebnis als JSON mit `file_put_contents()` speichern und zurückgeben. Die Anwendung füllt den Cache erst bei Bedarf.

08 **Invalidierung.** Einen veralteten Eintrag löschen oder als ungültig markieren. Nach einem Posterwechsel können Browser, CDN und Server noch unterschiedliche alte Kopien haben. Die Quelle ändert sich unabhängig von den Kopien, deshalb müssen alle betroffenen Ebenen berücksichtigt werden.

09 **Strategien. TTL** lässt Daten nach einer festen Zeit ablaufen, z. B. die Trending-Liste nach 5–15 Minuten. **Gezieltes Löschen** entfernt Einträge bei Änderungen, z. B. mit `unlink()`. **Versionierung** lädt statische Dateien unter neuer URL, z. B. `style.css?v=3`. Bei geänderter Filmbeschreibung alle Caches mit dieser Beschreibung invalidieren. Dazu gehören die Detailseite und je nach Inhalt Suche, Trending und „Neu hinzugefügt“.

10 **Personalisierte und volatile Daten.** Passwörter, Login-Daten und Tokens gehören in keinen gemeinsamen Cache. Watchlist und Profil müssen pro Person getrennt bleiben. Suchfilter oder zuletzt angesehene Titel lassen sich in der individuellen Session speichern. Schnell wechselnde Werte wie die Wiedergabeposition veralten im Cache rasch. Caching lohnt sich besonders für häufig gelesene und selten geänderte Daten.